



The Python Cheatsheet

elegant · visual · fun to flip through

variables

strings

lists

dicts

control flow

functions

classes

files

a one-stop reference for Python 3

by Cowork · v1.0

Variables & Strings

the building blocks · names, numbers, text

02 / 06



V Variables & Types

Names hold values. No type declarations needed.

```
name = 'Ada'          # str
age  = 36              # int
pi   = 3.14159         # float
ok   = True           # bool
x, y = 1, 2           # multi-assign
a = b = c = 0         # chain
```

Type checks & conversions

```
type(age)             # <class 'int'>
isinstance(age, int)  # True
int('42'); float('3.1'); str(7)
```

N Numbers & Math

```
5 + 2; 5 - 2; 5 * 2
5 / 2 # 2.5 true division
5 // 2 # 2 floor division
5 % 2 # 1 modulo
2 ** 10 # 1024 power
abs(-7); round(3.6); pow(2,8)
import math; math.sqrt(2)
```

S Strings

Immutable text. f-strings are your best friend.

```
s = 'hello'
s.upper()          # 'HELLO'
s.title()          # 'Hello'
s.replace('l', 'L') # 'heLlo'
s.split('e')       # ['h', 'llo']
'-'.join(['a', 'b']) # 'a-b'
len(s)             # 5
s[0]; s[-1]; s[1:4] # slice & index
```

f-strings

```
name, n = 'Ada', 7
f'hi {name}, {n*2} cookies'
f'pi = {pi:.2f}' # 'pi = 3.14'
f'{42:>5}'       # '   42'
```

Collections

lists · tuples · sets · dicts — pick the right shape

03 / 06



[list]



ordered · mutable

(tuple)



ordered · frozen

{ set }



unique · unordered

{ dict }



key → value

L Lists

```
xs = [3, 1, 4, 1, 5]
xs.append(9)      # add to end
xs.insert(0, 0)   # at index
xs.pop()          # remove last
xs.remove(1)      # first match
xs.sort(); xs.reverse()
xs[1:3]           # slice
len(xs); 4 in xs
```

D Dicts

```
d = {'name': 'Ada', 'age': 36}
d['email'] = 'a@x.io'
d.get('phone', 'n/a')
d.keys(); d.values()
d.items()         # pairs
for k, v in d.items(): ...
d.pop('age')
{'**d', 'role': 'admin'} # merge
```

T Tuples

```
p = (3, 4)        # frozen list
x, y = p           # unpack
p[0]              # 3
len(p); 3 in p
# returning multiple values
def split(s): return s, s.upper()
```

S Sets

```
a = {1, 2, 3}; b = {2, 3, 4}
a | b    # union      {1, 2, 3, 4}
a & b    # intersect  {2, 3}
a - b    # diff       {1}
a ^ b    # symdiff    {1, 4}
a.add(5); a.discard(2)
```

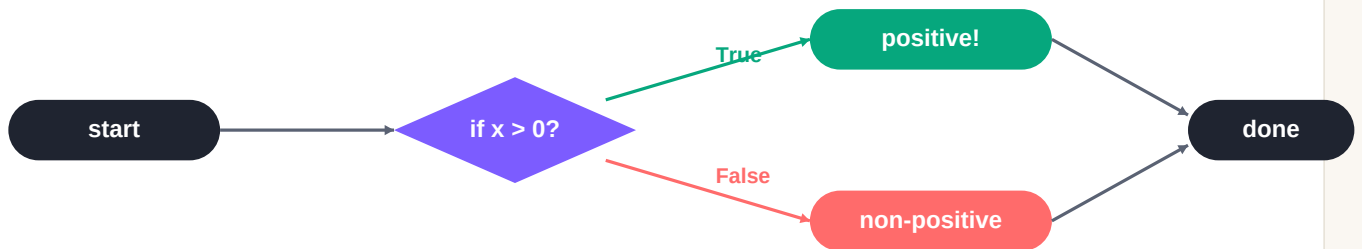
Control Flow

branches · loops · comprehensions

04 / 06



How a branch flows



? if / elif / else

```
if x > 0:
    print('pos')
elif x == 0:
    print('zero')
else:
    print('neg')

# ternary
label = 'even' if x%2==0 else 'odd'
```

Loops

```
for i in range(5):
    print(i)

for k, v in d.items():
    print(k, v)

while n > 0:
    n -= 1
    if n == 3: continue
    if n == 1: break
```

Comprehensions

```
[x*x for x in range(5)]
[x for x in xs if x > 0]
{w: len(w) for w in words}
{c for c in 'banana'}
```

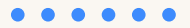
Σ enumerate · zip · range

```
for i, x in enumerate(xs): ...
for a, b in zip(xs, ys): ...
list(range(0, 10, 2))
# [0, 2, 4, 6, 8]
```

Functions & Classes

reusable logic · objects with behavior

05 / 06



f Functions

Define · call · return

```
def greet(name, loud=False):
    msg = f'hi {name}'
    return msg.upper() if loud else msg

greet('Ada')           # 'hi Ada'
greet('Ada', loud=True) # 'HI ADA'
```

```
def total(*nums, **opts):
    return sum(nums) * opts.get('k', 1)

double = lambda x: x * 2
```

@ Classes

Blueprints for objects

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        return f'{self.name}: woof!'

rex = Dog('Rex')
rex.bark() # 'Rex: woof!'

class Puppy(Dog):
    def bark(self): return 'yip!'
```

Anatomy of a class

```
class Dog:
```

```
    name : str
```

```
    __init__(self, name)
```

```
    bark(self) → str
```

→ class name — the blueprint

→ attributes — data per instance

→ constructor — runs on Dog(...)

→ methods — what an instance can do

Files, Errors & Toolkit

I/O · try/except · stdlib favorites

06 / 06



Files & I/O

```
with open('notes.txt') as f:
    text = f.read()

with open('out.txt', 'w') as f:
    f.write('hello')

import json
json.dump(d, open('d.json', 'w'))
```

Errors

```
try:
    n = int(s)
except ValueError as e:
    print('bad input:', e)
except (KeyError, IndexError):
    pass
else:
    print('all good')
finally:
    cleanup()
```

Stdlib Favorites

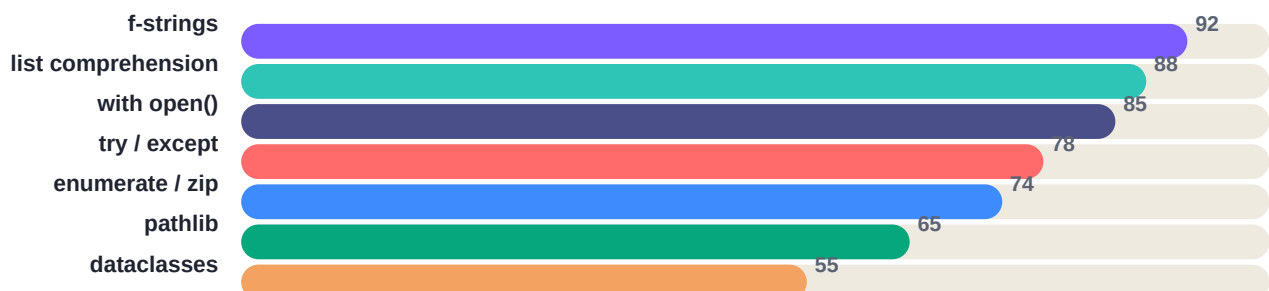
```
from pathlib import Path
Path('data').mkdir(exist_ok=True)
from collections import Counter
Counter('banana') # most_common
import datetime as dt
dt.date.today()
import random; random.choice(xs)
```

Virtual envs & pip

```
$ python -m venv .venv
$ source .venv/bin/activate
$ pip install requests pandas
$ pip freeze > requirements.txt
```

Pythonic habits to keep close

a friendly chart of things you'll reach for again and again



◆ keep this nearby — and write more Python ◆